

Семинар 1-2

- Особенности языка Rust
- Базовые синтаксис
- Разработка CLI



Особенности Rust

Безопасность без потери эффективности и с повышением продуктивности!

- Безопасная работа с памятью → отсутствие утечек памяти и переполнения буфера
- Отсутствие сборщика мусора → предсказуемая производительность
- Реально нужный отладочный режим
 - Самые понятные сообщения об ошибках в истории ЯП
- Перенос многих проверок на уровень компиляции
- Многопоточность без страха и эффективное распараллеливание
- Современные строки и коллекции в ядре языка → высокоуровневые абстракции и АТД
- Система сборки и пакетирования здорового человека

White House Office urges memory safety (<https://stackoverflow.blog/2024/12/30/in-rust-we-trust-white-house-office-urges-memory-safety/>)

Ошибка на миллион долларов

I call it my billion-dollar mistake. It was the invention of the null reference in 1965. This has led to innumerable errors, vulnerabilities, and system crashes, which have probably caused a billion dollars of pain and damage in the last forty years

- Tony Hoare, 2009

Продуктивность в Rust в больших кодовых базах

- Компилятор Rust позволяет отловить множество распространенных ошибок на этапе компиляции, что удобно при работе с большим проектом
- За это Rust расплачивается длительным временем компиляции
 - Можно ускорить убрав флагами часть проверок
- Rust в Google: превосходит по продуктивности C++ и сравнялся с Go (<https://www.ardanlabs.com/news/2024/rust-at-google/>)
 - Об этом говорят и владельцы более мелких кодовых баз (<https://lubeno.dev/blog/rusts-productivity-curve>)

Иллюстративные примеры переезда на Rust

- Cloudflare переехал с nginx на Pingora
 - ~70% меньше CPU и ~67% меньше памяти на том же трафике
 - Обслуживает свыше 1 триллиона запросов в день, 40+ млн req/s
 - Убраны целые классы багов: use-after-free, double-free и тп
 - Rspack - замена webpack на Rust с сохранением API
 - мигрировали: Discord, Mews
 - uv - замена pip и один из лучших менеджеров пакетов для Python
- Продуктивный программист на Rust Программист на другом языке



История переезда bun с Zig на Rust

- 2 декабря 2025 — Anthropic покупает Bun; проект остаётся open source
- 14 мая 2026 — версия 1.3.14 объявлена последней на Zig. В тот же день в main мержится PR на более миллиона строк Rust
- Перенос сделан AI-ассистированно (Jared Samner использовал предрелизную Fable 5). За пять дней до мержа проходило 99.8% старого тест-сюита на Linux x64, к мержу — все платформы
- Итог: бинарник на 3–8 МБ меньше, производительность «нейтральная или лучше», а use-after-free и double-free стали ошибками компиляции. Samner подчёркивает: это «тот же кодбейс, портированный на Rust», архитектура и структуры данных не менялись
- Текущая версия — 1.4.2 (5 сентября 2026), проект продолжает активно развиваться, но остался вопрос, это unreviewed slop или пример отличного использования LLM?

Основы синтаксиса и семантики языка

- **Область видимости [Scope]**
 - Различают как минимум область видимости переменной (из-за Non-lexical lifetimes), блока, референта
- **Оператор [Statment]** — независимая единица управления потоком выполнения
 - ";" бывает как разделителем, так и завершителем
- **Выражение [Expresion]** — фраза языка, возвращающая значение
 - Может вызывать побочные эффекты, но нужно постараться
- **Предикат [Predicat]** — выражение, которое возвращает значение типа, совместимого с логическим
 - Используются в качестве условий

Переменные

- Переменные по умолчанию неизменяемые
- Изменяемые переменные помечаются `mut`
 - ! С учётом последующего владения

```
1 fn main() {  
2     let x = 5;  
3     println!("The value of x is: {x}");  
4     let mut y = 7;  
5     y = 6;  
6     println!("The value of x is: {x}");  
7     {  
8         let x = x * 2;  
9         println!("The value of x is: {x}");  
10    }  
11 }
```

Ключевые слова Rust

- Ключевые слова (КС) бывают трех видов:
 - **strict** - нельзя переопределять и использовать вне определенного контекста
 - **reserved** - нельзя переопределять, зарезервированы на будущее
 - **weak** - можно переопределять в зависимости от контекста
- Strict keywords:

as	dyn	if	move	static	use
async	else	impl	mut	struct	where
await	enum	in	pub	super	while
break	extern	let	ref	trait	
const	false	loop	return	true	
continue	fn	match	self	type	

Типы данных (1)

- Целые числа (Integer types)

Length	Signed	Unsigned
8-bit	i8	u8
16-bit	i16	u16
32-bit	i32	u32
64-bit	i64	u64
128-bit	i128	u128
arch depend	isize	usize

Типы данных (2)

- Числа с плавающей запятой (Floating-point types)

```
1 let x = 2.0; // f64
2 let y: f32 = 3.0; // f32
```

- Логические значения (Boolean type)

```
1 let t = true;
2 let f: bool = false; // with explicit type annotation
```

- Символ (Character type)

```
1 let c = 'z';
2 let z: char = 'Z'; // with explicit type annotation
3 let heart_eyed_cat = '😻';
```

Типы данных (3)

- Массив (Array type)

```
let a: [i32; 5] = [1, 2, 3, 4, 5];  
let first = a[0];  
let second = a[1];
```

- Картеж (Tuple type)

```
let tup = (500, 6.4, 1);  
println!("The value of y is: {tup.1}");  
  
let (x, y, z) = tup;  
println!("The value of y is: {y}");
```

Функции

- Если последним оператором тела функции является выражение без ";" [ExpresionWithoutBlock], то значение этого выражения — результат функции
- Возможно использование КС `return` для завершения функции

```
1 fn plus_one(x: i32) -> i32 {  
2     x + 1  
3 }  
4  
5 fn print_x(x: i32) {  
6     println!("The value of x is: {x}");  
7 }
```

Комментарии

```
1 /// Adds one to the number given.
2 /// # Examples
3 /// ```
4 /// let arg = 5;
5 /// let answer = my_crate::add_one(arg);
6 ///
7 /// assert_eq!(6, answer);
8 /// ```
9 pub fn add_one(x: i32) -> i32 {
10     // TODO: Think why I actually need this
11     x + 1
12     /*
13         ...
14         A lot of lines
15         ...
16     */
17 }
```

if - else

```
1 fn main() {  
2     let number = 6;  
3  
4     if number % 4 == 0 {  
5         println!("number is divisible by 4");  
6     } else if number % 3 == 0 {  
7         println!("number is divisible by 3");  
8     } else if number % 2 == 0 {  
9         println!("number is divisible by 2");  
10    } else {  
11        println!("number is not divisible by 4, 3, or 2");  
12    }  
13 }
```

Цикл loop и маркеры

```
fn main() {  
    let mut count = 0;  
    'counting_up: loop {  
        let mut remaining = 10;  
        loop {  
            if remaining == 9 {  
                break;  
            }  
            if count == 2 {  
                break 'counting_up;  
            }  
            remaining -= 1;  
        }  
        count += 1;  
    }  
}
```

```
fn main() {  
    let mut counter = 0;  
    let result = loop {  
        counter += 1;  
  
        if counter == 10 {  
            break counter * 2;  
        }  
    };  
    println!("Result: {result}");  
}
```

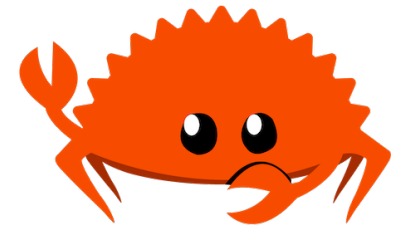
Цикл while

```
1 fn main() {  
2     let a = [10, 20, 30, 40, 50];  
3     let mut index = 0;  
4  
5     while index < 5 {  
6         println!("the value is: {}", a[index]);  
7         index += 1;  
8     }  
9 }
```

Правила владения

- Каждое значение в Rust имеет своего владельца
- В конкретный момент у каждого значения может быть только один владелец
- Когда владелец уходит из области видимости все чем он владеет уничтожается

```
let s1 = String::from("hello");  
let s2 = s1;  
println!("{s1}, world!"); // compiling error
```



Ссылки и заимствования

- **Ссылочный тип** [Pointer type], далее будем называть **ссылкой**
- В любой момент времени у вас может быть либо одна мутабельная ссылка, либо любое количество немутабельных ссылок
- Ссылки всегда должны быть валидными

```
1 fn calculate_length(s: &String) -> usize {  
2     s.len()  
3 }  
4 fn change(some_string: &mut String) {  
5     some_string.push_str(", world");  
6 }  
7 fn dangle() -> &String { // dangle returns a reference to a String  
8     let s = String::from("hello");  
9     &s  
10 }
```

Тип среза [Slice Type]

- На жаргоне будем называть переменные этого типа **слайсами**

```
1 let s = String::from("hello world");
2 let hello = &s[0..5];
3 let world = &s[6..11];
4
5
6 let a = [1, 2, 3, 4, 5];
7 let slice = &a[1..3];
8 assert_eq!(slice, &[2, 3]);
9
10 let s = String::from("hello");
11 let len = s.len();
12 let slice = &s[3..len];
13 let slice = &s[3..];
```

Структуры

```
1 struct User {
2     email: String,
3     sign_in_count: u64,
4 }
5 fn build_user(email: String) -> User {
6     User {
7         email,
8         sign_in_count: 1,
9     }
10 }
11 fn main() {
12     let user1 = build_user("some@example.com");
13     let user2 = User {
14         email: String::from("another@example.com"),
15         ..user1
16     };
17 }
```

Структуры-кортежи и структуры без полей

```
1 struct Color(i32, i32, i32);
2 struct Point(i32, i32, i32);
3
4 struct AlwaysEqual;
5
6 fn main() {
7     let black = Color(0, 0, 0);
8     let origin = Point(0, 0, 0);
9     let subject = AlwaysEqual;
10
11 }
```

Реализация методов структур

```
1 #[derive(Debug)]
2 struct Rectangle {
3     width: u32,
4     height: u32,
5 }
6
7 impl Rectangle {
8     fn area(&self) -> u32 {
9         self.width * self.height
10    }
11 }
12
13 fn main() {
14     let rect1 = Rectangle { width: 30, height: 50, };
15     println!("The area {}", rect1.area());
16 }
```

Перечисления

```
1 enum Message {  
2     Quit,  
3     Move { x: i32, y: i32 },  
4     Write(String),  
5     ChangeColor(i32, i32, i32),  
6 }
```

```
1 struct QuitMessage; // unit struct  
2 struct MoveMessage {  
3     x: i32,  
4     y: i32,  
5 }  
6 struct WriteMessage(String); // tuple struct  
7 struct ChangeColorMessage(i32, i32, i32); // tuple struct
```

Match и сопоставление с шаблоном

- `match` обязывает обработать все варианты значения перечисления
- Подробнее это мощнейший конструкт рассмотрим позднее

```
1 fn action(message: Message) {  
2     match message {  
3         Message::Quit => println!("Quit"),  
4         Message::Move { x, y } => println!("Move to {x} {y}"),  
5         Message::Write(m) => println!("{m}"),  
6         Message::ChangeColor(r, g, b) =>  
7             println!("Change color to {r} {g} {b}"),  
8     }  
9 }
```

Пример использования match с другими типами

```
1 let dice_roll = 9;
2 match dice_roll {
3     3 => add_fancy_hat(),
4     7 => remove_fancy_hat(),
5     _ => reroll(),
6 }
7
8 fn add_fancy_hat() {}
9 fn remove_fancy_hat() {}
10 fn reroll() {}
```

if let - else

```
1 let message = Message::Move { x: 7, y: 6 };
2 if let Message::Move{x, y} = message {
3     println!("Move to {x} {y}");
4 } else {
5     println!("Not move")
6 }
```

```
1 let mut message = Message::Move {x: 7, y: 6};
2 while let Message::Move{x, y} = message {
3     if x >= 10 {
4         message = Message::Quit;
5     } else {
6         message = Message::Move {x: x+1, y};
7     }
8 }
```

Optional

```
1 enum Option<T> {  
2     None,  
3     Some(T),  
4 }
```

```
1 let some_number = Some(5);  
2 let some_char = Some('e');  
3  
4 let absent_number: Option<i32> = None;
```

```
1 let x: i8 = 5;  
2 let y: Option<i8> = Some(5);  
3  
4 let sum = x + y; // compiling error
```

Работа с ошибками

- Невосстановимые ошибки

```
1 fn main() {  
2     panic!("crash and burn");  
3 }
```

- Восстановимые ошибки (пример на следующем слайде)

```
1 enum Result<T, E> {  
2     Ok(T),  
3     Err(E),  
4 }
```

Распространение ошибок (1)

```
1 use std::fs::File;
2 use std::io::{self, Read};
3
4 fn read_username_from_file() -> Result<String, io::Error> {
5     let username_file_result = File::open("hello.txt");
6     let mut username_file = match username_file_result {
7         Ok(file) => file,
8         Err(e) => return Err(e),
9     };
10    let mut username = String::new();
11    match username_file.read_to_string(&mut username) {
12        Ok(_) => Ok(username),
13        Err(e) => Err(e),
14    }
15 }
```

Распространение ошибок (2)

- "?" как синтаксический сахар

```
1 use std::fs::File;
2 use std::io::{self, Read};
3
4 fn read_username_from_file() -> Result<String, io::Error> {
5     let mut username_file = File::open("hello.txt");
6     let mut username = String::new();
7     username_file.read_to_string(&mut username)?;
8     Ok(username)
9 }
```

Unwrap

- Паникуем, если не смогли достать значение из `Option` и правильный результат из `Result`
- Удобный механизм для прототипирования и забивания на ошибки
- Стоит избегать в продакшн-коде

```
1 let a = Some(5);  
2 let b = a.unwrap() + 5;  
3 println!("{b}");
```

```
1 let a: Result<i8, ParseError> = Ok(3);  
2 let b = a.unwrap() + 5;  
3 println!("{b}");
```

Стандартные коллекции: Вектор

```
1 let v = vec![100, 32, 57];  
2 for i in &v { println!("{i}"); }
```

```
1 let mut v = vec![1, 2, 3, 4, 5];  
2 let first = &v[0];  
3 v.push(6);  
4 println!("The first element is: {first}");
```

```
1 let third: Option<i32> = v.get(2);  
2 match third {  
3     Some(third) => println!("The third element is {third}"),  
4     None => println!("There is no third element."),  
5 }
```

Храним разные типы лучше, чем Python

```
1 enum SpreadsheetCell {  
2     Int(i32),  
3     Float(f64),  
4     Text(String),  
5 }  
6  
7 let row = vec![  
8     SpreadsheetCell::Int(3),  
9     SpreadsheetCell::Text(String::from("blue")),  
10    SpreadsheetCell::Float(10.12),  
11 ];
```

String, храним текст в UTF-8

```
1 let mut s = String::from("こんにちは");
2 s.push_str("bar");
```

- `&str != String`

```
1 "blue"; // &str
2 "red".to_string(); // String
3 String::from("hi"); // String
4 "rust is fun!".to_owned(); // String
5 "nice weather".into(); // String
6 format!("Interpolation {}", "Station"); // String
7 &String::from("abc")[0..1]; // &str
8 " hello there ".trim(); // &str
9 "Happy Monday!".replace("Mon", "Tues"); // String
10 "mY sHiFt KeY iS sTiCkY".to_lowercase(); // String
```

UTF-8 Everywhere (<http://utf8everywhere.org/>)

Стандартные коллекции: HashMap

```
1 use std::collections::HashMap;
2
3 let mut scores = HashMap::new();
4
5 scores.insert(String::from("Blue"), 10);
6 scores.insert(String::from("Yellow"), 50);
7
8 let team_name = String::from("Blue");
9 let score = scores.get(&team_name).copied().unwrap_or(0);
10
11 for (key, value) in &scores {
12     println!("{key}: {value}");
13 }
14
15 let new_team_name = String::from("Red");
16 scores.insert(new_team_name, score); // take ownership of new_team_name
```

Обобщённые методы (дженерики)

```
1 fn largest<T>(list: &[T]) -> &T {
2     let mut largest = &list[0];
3     for item in list {
4         if item > largest {
5             largest = item;
6         }
7     }
8     largest
9 }
10 fn main() {
11     let number_list = vec![34, 50, 25, 100, 65];
12     let result = largest(&number_list);
13
14     let char_list = vec!['y', 'm', 'a', 'q'];
15     let result = largest(&char_list);
16 }
```

Дженерик-структуры

```
1 struct Point<T> {  
2     x: T,  
3     y: T,  
4 }  
5 impl<T> Point<T> {  
6     fn x(&self) -> &T { &self.x }  
7 }  
8 impl Point<f32> {  
9     fn distance_from_origin(&self) -> f32 {  
10         (self.x.powi(2) + self.y.powi(2)).sqrt()  
11     }  
12 }  
13 fn main() {  
14     let integer = Point { x: 5, y: 10 };  
15     let float = Point { x: 1.0, y: 4.0 };  
16 }
```

Дженерик-перечисления

```
1 enum Option<T> {  
2     Some(T),  
3     None,  
4 }
```

```
1 enum Result<T, E> {  
2     Ok(T),  
3     Err(E),  
4 }
```

Трейты [Trait]

```
1 trait Summary {  
2     fn summarize(&self) -> String;  
3 }  
4  
5 struct NewsArticle {  
6     headline: String,  
7     author: String,  
8 }  
9  
10 impl Summary for NewsArticle {  
11     fn summarize(&self) -> String {  
12         format!("{}", by {} ({}), self.headline, self.author)  
13     }  
14 }
```

Использование Trait (1)

- Trait параметр

```
fn notify(item: &impl Summary) {  
    println!("Breaking news! {}", item.summarize());  
}
```

- Trait Bound Syntax

```
fn notify<T: Summary>(item: &T) {  
    println!("Breaking news! {}", item.summarize());  
}
```

- Несколько Trait Bound

```
fn notify<T: Summary + Display>(item: &T) {}
```

Использование Trait (2)

- Возвращаем тип, реализующий Trait

```
1 fn returns_summarizable() -> impl Summary {  
2     NewsArticle {  
3         headline: String::from("Rust is cool"),  
4         author: String::from("Alex Glushko")  
5     }  
6 }
```

- Делаем код чище с помощью КС where

```
1 fn some_function<T, U>(t: &T, u: &U) -> i32  
2 where  
3     T: Display + Clone,  
4     U: Clone + Debug,  
5 {}
```

Исправляем функцию `largest`

```
1 fn largest<T: PartialOrd>(list: &[T]) -> &T {
2     let mut largest = &list[0];
3     for item in list {
4         if item > largest {
5             largest = item;
6         }
7     }
8     largest
9 }
10
11 fn main() {
12     let number_list = vec![34, 50, 25, 100, 65];
13     let result = largest(&number_list);
14
15     let char_list = vec!['y', 'm', 'a', 'q'];
16     let result = largest(&char_list);
17 }
```

Что нужно в качестве пререквизитов для эффективного управления ЖЦ данных

- Устройство памяти современного цифрового компьютера
 - Главное: отличать стек от кучи
- Типы значения и типы ссылки [Value / Reference Types]
- Распаковка и упаковка [Boxing / UnBoxing]
- Жизненный цикл (ЖЦ) [Lifetime]
 - Включая NLL

Модули: базовый синтаксис

```
1 mod front_of_house {  
2     pub mod hosting {  
3         pub fn add_to_waitlist() {}  
4     }  
5 }  
6  
7 pub use crate::front_of_house::hosting;  
8  
9 pub fn eat_at_restaurant() {  
10     hosting::add_to_waitlist();  
11 }
```

Внешние зависимости

- Добавляем командой `cargo add name_of_crate`
e.g. `cargo add rand`

```
1 use rand::Rng;
2
3 fn main() {
4     let secret_number = rand::thread_rng().gen_range(1..=100);
5 }
```

- Чистим большие списки импорта

```
1 use std::{cmp::Ordering, io}; // nested path
2 use std::io::{self, Write}; // nested path with self
3 use std::collections::*; // global operator
```

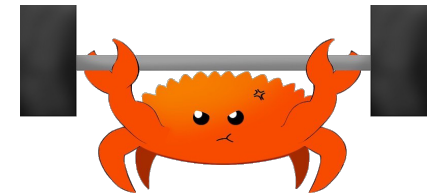
Пишем свой первый CLI

Подготовка:

- Создаем проект `cargo new proj_name`
- Добавляем пакет `clap` для работы с аргументами командной строки

Задание:

- Разработать CLI для преобразования файл с Python словарем в JSON



Пример файла

- До

```
{  
  'some': 3,  
  'other': 'text'  
}
```

- После

```
{  
  "some": 3,  
  "other": "text"  
}
```

Пример использования clap

```
1 use clap::Parser;
2
3 #[derive(Parser)]
4 #[command(version = "0.1.0",
5     about = "Some short", long_about = "Some long")]
6 struct Cli {
7     arg: String,
8     #[arg(short, long)]
9     optional_arg: Option<String>,
10 }
11 fn main() {
12     let args = Cli::parse();
13     println!("Arg: {}", args.arg);
14     if let Some(arg) = args.optional_arg {
15         println!("Optional: {arg}");
16     }
17 }
```

Пример чтения и записи в файл

```
1 use std::fs::File;
2 use std::io::{Write, BufReader, BufRead, Error};
3
4 fn main() -> Result<(), Error> {
5     let path = "lines.txt";
6     let mut output = File::create(path)?;
7     write!(output, "Rust\n💖\nFun")?;
8     let input = File::open(path)?;
9     let buffered = BufReader::new(input);
10    for line in buffered.lines() {
11        println!("{}", line?);
12    }
13    Ok(())
14 }
```